Introducing pharo-agentic-browser

Multi-Agent Session Manager for Pharo Smalltalk

Masashi Umezawa

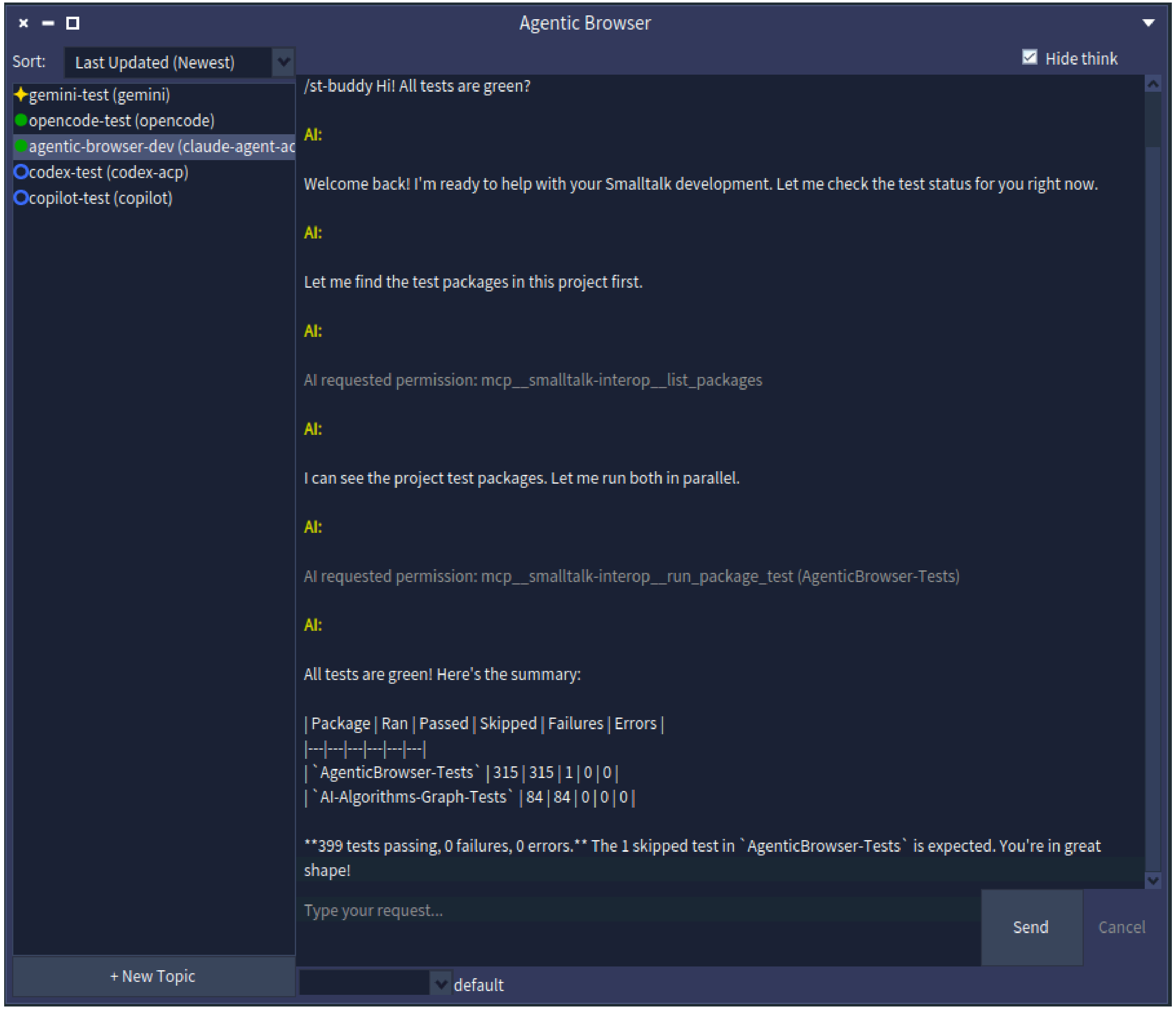
https://github.com/mumez/pharo-agentic-browser

What is pharo-agentic-browser?

pharo-agentic-browser

> A **Pharo-native GUI tool** for managing multiple AI coding agent sessions — Claude Code, Gemini CLI, OpenCode, and others — in parallel from inside your Pharo image.

At a Glance


Agentic Browser
Sort:
Last Updated (Newest)
Hide think
gemini-test (gemini)
opencode-test (opencode)
agentic-browser-dev (claude-agent-ac
codex-test (codex-acp)
copilot-test (copilot)
/st-buddy Hi! All tests are green?
AI:
Welcome back! I'm ready to help with your Smalltalk development. Let me check the test status for you right now.
AI:
Let me find the test packages in this project first.
AI:
AI requested permission: mcp__smalltalk-interop__list_packages
AI:
I can see the project test packages. Let me run both in parallel.
AI:
AI requested permission: mcp__smalltalk-interop__run_package_test (AgenticBrowser-Tests)
AI:
All tests are green! Here's the summary:
Package	Ran	Passed	Skipped	Failures	Errors
`AgenticBrowser-Tests`	315	315	1	0	0
`AI-Algorithms-Graph-Tests`	84	84	0	0	0
399 tests passing, 0 failures, 0 errors. The 1 skipped test in `AgenticBrowser-Tests` is expected. You're in great shape!
Type your request...
Send
Cancel
+ New Topic
default

Core Workflow

Each session is called a **topic**:

1. Create a topic and select an ACP-compatible agent
2. Type a request + mention code with `@ClassName` or attach a screenshot
3. The AI works autonomously (task decomposition, code changes, tests)
4. When the AI needs approval, it pauses and asks in the chat
5. Respond with pulldown selection — the AI resumes
6. Topic status is always visible in the sidebar

Motivation

AI GUI Tools Are Going Mainstream

Dedicated GUI tools for AI coding agents are becoming standard:

- **Claude Desktop** — Claude with tools, MCP, and file access
- **Codex Desktop** — OpenAI's autonomous coding environment
- **Cursor / Windsurf** — AI-native editors

These tools lower the barrier for interacting with AI agents beyond simple chat.

The Shift: IDE → Multi-Agent Delegation

Era	Paradigm	Examples
Early	Chat in editor sidebar	ChatGPT, GitHub Copilot
Now	AI-first IDE with agent mode	Cursor, Windsurf
Emerging	**Delegate full tasks to agents**	Antigravity 2.0, Codex Desktop

The AI can now handle **entire feature-level tasks**, not just line completions.

Tools are evolving: the UI is no longer "editor + chat" — it's **session orchestration**.

Why Pharo Needs This Too

Pharo developers should be able to leverage the same paradigm:

- **Rich live environment** — classes, methods, and runtime are right there
- **pharo-acp** already provides ACP client support for multiple agents
- **Multiple projects** can run in parallel in one image

> A Pharo-native GUI that can control multiple AI agents is the natural next step.

Key Advantages Over Generic Tools

Advantage	Details
Direct context passing	`@ClassName`, `@Class>>method`, screenshot — no copy-paste
Agent-agnostic	Works with any ACP agent via pharo-acp
Pharo-integrated	Tests, code export, change watching all use the live image
Parallel sessions	Run several agents on different topics simultaneously

Installation

Installation — Pharo Side

Open a Playground in a Pharo 12+ image and evaluate:

```
Metacello new
    baseline: 'AgenticBrowser';
    repository: 'github://mumez/pharo-agentic-browser:main/src';
    load.
```

Then open the browser:

```
AgenticBrowser open.
```

Installation — Agent Side

Any ACP-compatible agent - preset list:

Agent	Install
Claude Code	`npm install -g @agentclientprotocol/claude-agent-acp`
Codex	`npm install -g @agentclientprotocol/codex-acp`
Gemini CLI	ACP built-in (`gemini --acp`)
Copilot CLI	ACP built-in (`copilot --acp --stdio`)
Cursor CLI	ACP built-in (`agent acp`)

Also supported: OpenCode, Kilo Code, and more

> ***Recommended***: *Install smalltalk-dev-plugin in your agent for better Smalltalk*

Basic Features

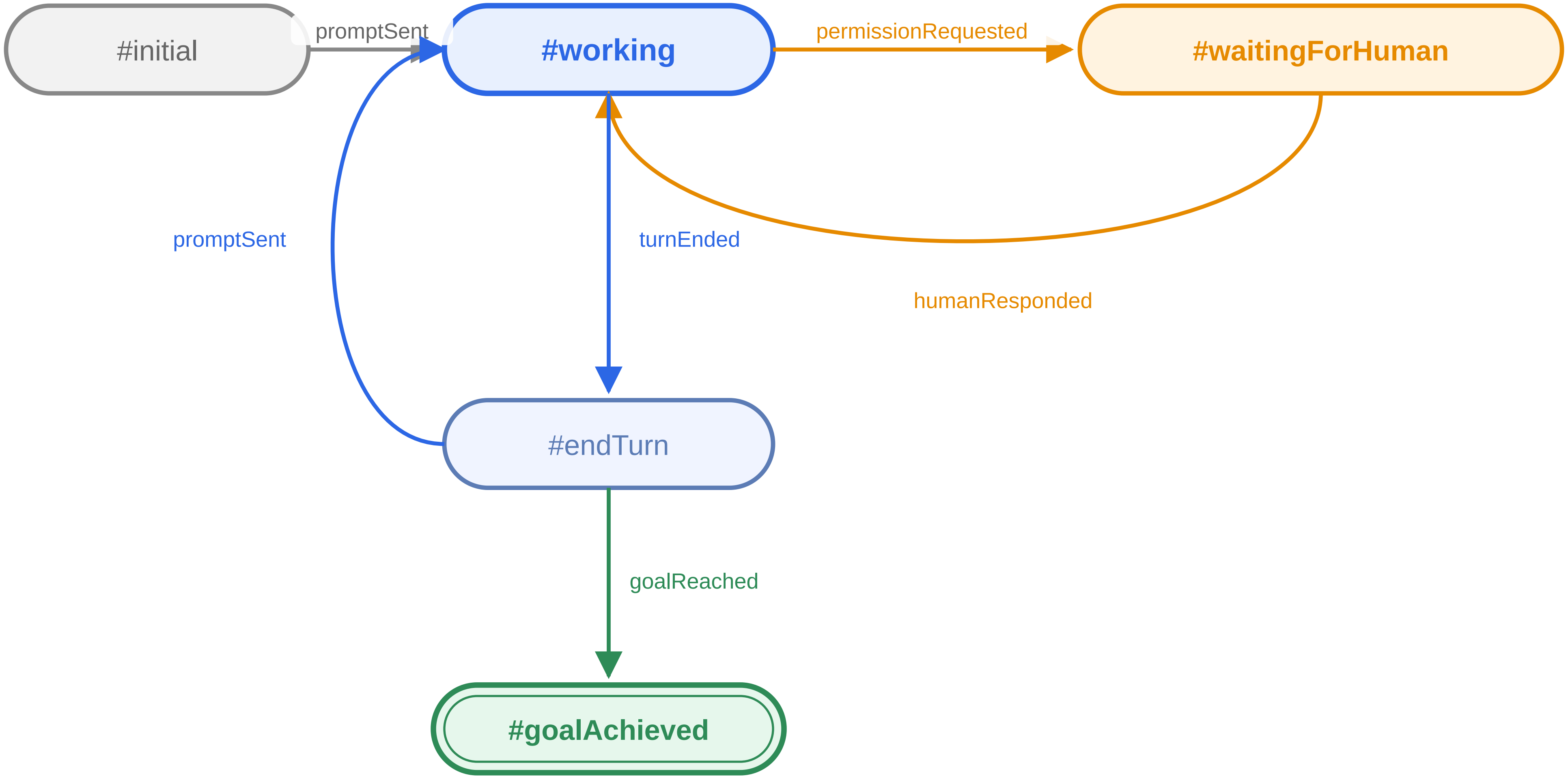
Creating a Topic

1. Click **+ New Topic**
2. Enter a title and select an agent
3. (Optional) Set an existing project directory
4. Click **Create** — topic appears in the left sidebar
5. (Optional) Right-click → **Set Target Packages...** to configure tracked packages

> The first message is automatically prefixed with `/st-buddy` to activate the Smalltalk buddy agent mode.

Topic States

Each topic follows a clear state machine:



Topic State Icons

Icon	State	Meaning
❇️	working	AI is actively running
?	waitingForHuman	AI needs approval
●	endTurn	Turn completed
✓	goalAchieved	Goal reached

Human-in-the-Loop

When the AI requests permission:

- Status changes to `?` (waitingForHuman)
- **Send** button becomes **Confirm**
- **Cancel** button becomes **Deny**

Respond by clicking the button — the AI resumes seamlessly.

> No modal dialogs. Approval is part of the conversation flow.

Code Mentions

Reference Pharo classes or methods directly in the chat:

```
@QueryClass @DBAdapter>>connect please refactor this
```

AgenticBrowser resolves each `@mention` to its Tonel source and attaches it as an ACP text resource — **no copy-paste needed**.

Drag and Drop

- Drag a **class** from the System Browser → inserts `@ClassName`
- Drag a **method** from the System Browser → inserts `@ClassName>>methodName`

Screen Captures

Click the `[ ]` button to attach a screenshot:

1. Click the button — cursor becomes a crosshair
2. Drag to select the area
3. A mention like `@sc-20260528-001.png` is inserted in the input
4. Send — the PNG is attached as an image resource to the prompt

Files are saved to `<agenticBrowserRoot>/screenshots/sc-YYYYMMDD-NNN.png`.

Goal Setting

Right-click a topic → **Set Goal...** to enter a completion condition:

```
all tests pass
```

AgenticBrowser sends a goal prompt to the AI:

> *Goal has been set: all tests pass. When the goal is achieved, summarize and report in result-<topic-id>.md. Keep retrying until the goal is achieved.*

When `result-<topic-id>.md` is created, the topic transitions to `✓` (`#goalAchieved`).

Goal Achievement Hooks

Two hooks fire when a goal is reached:

Announcement

```
topic announcer
    when: AbTopicGoalAchieved
    do: [:ann | Transcript crShow: ann topic title , ' achieved: ' , ann goal result].
```

Callback Block

```
topic whenGoalAchieved: [:goal | Transcript crShow: goal result].
```

Integrate goal events into your own automation workflows.

Session Persistence

Topics are automatically saved to `ab-topics.fuel` using Pharo's **Fuel** serializer — they survive image restarts.

Manual save/restore:

```
AbTopicManager save.
AbTopicManager load.
```

Per-topic state (settings, status, conversation) is fully persisted.

Customization

Adding MCP Servers

Place `mcp.json` in your AgenticBrowser root directory:

```json
{
  "mcpServers": {
    "my-server": {
      "command": "uvx",
      "args": ["my-server-package"],
      "env": {"API_KEY": "value"}
    }
  }
}
```

- Built-in `smalltalk-interop` and `smalltalk-validator` servers are **auto-merged** by default
- Set `useDefaultMcpServers: false` to use only your own `mcp.json`

Adding Custom Agents

From a Playground

```
AbSettings default codingAgents: (AbSettings default codingAgents copyWith:
    {'name' -> 'my-agent'.
    'command' -> #('my-agent' '--acp')} asDictionary).
AbSettings save.
```

Or edit `ab-settings.json` directly

The agent appears as a preset in the **New Topic** dialog on next open.

Global Settings

Key	Default	Description
`useDefaultMcpServers`	`true`	Merge built-in Smalltalk MCP servers
`aiPermissionWaitTimeoutSeconds`	`1800`	Timeout for human approval
`aiPermissionTimeoutOption`	`#reject_once`	Auto-response: `allow_once`, `allow_always`, `reject_once`
`exportApprovalWaitTimeoutSeconds`	`30`	Timeout for package export approval

Settings can also be configured **per-topic** via right-click → **Edit Settings...**

Summary

Summary

pharo-agentic-browser brings the multi-agent delegation paradigm to Pharo:

- **Native GUI** — manage multiple AI sessions without leaving your image
- **Agent-agnostic** — any ACP-compatible agent works
- **Rich context** — code mentions, drag-and-drop, screen captures
- **Human-in-the-loop** — conversational approval, no interruption dialogs
- **Goal-driven** — set completion conditions, hook into results
- **Extensible** — custom MCP servers and agents with minimal configuration

Feedback and contributions are welcome!

https://github.com/mumez/pharo-agentic-browser